Как текст представлен в C?

Символьный тип

- **char** - ещё один фундаментальный тип данных в языке C
- Используется для хранения одиночных символов. Сами символы указываются в **одинарных** кавычках, например: 'a', 'b', 'd', '4', '%', '!'
- Символы хранятся в виде **целых чисел**. Фактически, **char** является ещё одним целочисленным типом
- Это единственный тип, размер которого ограничен стандратом языка. Размер **char** **всегда** равен 1 байту
- Но всё просто не бывает: стандарт не обговаривает, должен ли тип **char** быть знаковым или беззнаковым
- Существуют специальные символы, которые в текстовом виде представлены более чем одним знаком: '\n', '\t', '\r'
- Также стандартом определены беззнаковый **unsigned char** и знаковый **signed char** типы

Символы и кодировка

Кодировка - специальная таблица, связывающая каждый символ с соответствующим ему целым числом
Базовая для ЭВМ - **ASCII**. Как правило, входит в любую другую расширенную кодировку. Основные характеристики

- 128 символов
- целочисленные коды: от нуля до 127
- каждый символ занимает один байт
- Включает в себя все цифры, буквы английского алфавита в нижнем/верхнем регистрах и некоторые другие символы (тильда, процент, '@', решётка и т.п.)
- Коды букв и цифр - идут последовательно

Символьный тип

```
1  char symbol = '%';
2  putchar(symbol); // печатаем знак процента
3
4  symbol = '#'; // знак переноса строки
5  printf("\nВыводим символ %c %c и ещё раз: %c↩
       \n",
6          symbol, '\n', symbol);
7
8  // А поскольку char — целочисленный:
9  symbol = '5';
10 symbol += 2;
11
12 // Выводим на экран семёрку
13 putchar(symbol);
```

Символьный тип

```
1  char sym = '5';
2  printf("Значение sym: %c\n", sym);
3
4  printf("Код sym: %d", sym);
5
6  int is_less = '2' < '5';
7  // Переменная is_less здесь равна 1
8
9  // Печать всех букв английского алфавита
10 sym = 'a';
11
12 while (sym <= 'z') {
13   printf("%c ", sym);
14   ++sym;
15 }
```

Символы и кодировка

Ещё одна кодировка, про которую полезно знать:
UTF8

- символ может состоять от 1 до 4 байт (следовательно, от 8 до 32 бит)
- вводится понятие "code point равное 1 байту (8 битам) символа
- включает в себя все возможные региональные алфавиты
- Коды от 0 до 127 совпадают с кодировкой ASCII

С-строки

Строка в С - это одномерный массив значений типа **char**, последним символом которой считается специальный нулевой символ (он же - символ окончания строки): \0 (целочисленный код - 0).

ПРЕДУПРЕЖДЕНИЕ

Не путайте '\0', '0' и "0".

Первое – символьный литерал, соответствующий символу с кодом 0.

Второе – такой же литерал, но обозначающий цифру 0, ее код в ASCII-кодировке 48.

Третий — это строковый литерал, содержащий два символа, цифру 0 и нуль-терминатор.

С-строки

В программе строки могут определяться следующим образом:

- как строковые константы;
- через указатель на символьный тип;
- как массивы символов;
- как массивы строк.

Кроме того, должно быть предусмотрено выделение памяти для хранения строки.

С-строки

Любая последовательность символов, заключенная в двойные кавычки «», рассматривается как строковая константа.
Для помещения в строковую константу некоторых служебных символов используются символьные комбинации.
Так, если необходимо включить в строку символ двойной кавычки, ему должен предшествовать символ «обратный слеш»:

```
'\"'.
```

Строковые константы размещаются в статической памяти.
Начальный адрес последовательности символов в двойных кавычках трактуется как адрес строки. Строковые константы часто используются для осуществления диалога с пользователем в таких функциях, как printf().

С-строки

При определении массива символов необходимо сообщить компилятору требуемый размер памяти.

```
1 char m[82];
```

Компилятор также может самостоятельно определить размер массива символов, если инициализация массива задана при объявлении строковой константой:

```
1 char m2[]="To be or not to be";
```

В этом случае имена m2 и m3 являются указателями на первые элементы массивов:

```
1     m2      // эквивалентно &m2[0]
2     m2[0]   // эквивалентно 'T'
3     m2[1]   // эквивалентно 'o'
```

Обратите внимание, символ конца строки **не обязан совпадать** с последним элементом массива.

```
1  char phrase1[] = "Hello!";
2  // puts — функция из stdio.h для
3  // вывода строки на консоль
4  puts(phrase1);
5
6  char phrase2[] = { 'A', 'B', 'C', '\0' };
7  printf("Второй символ в phrase2: %c",
8         phrase2[1]);
9
10 // Массив, который пока строкой не является
11 char test_str[15];
```

При объявлении массива символов и инициализации его строковой константой можно явно указать размер массива, но указанный размер массива должен быть больше, чем размер инициализирующей строковой константы:

```
1 char m2[8]="To be or not to be"; // ОШИБКА
2 char m2[80]="To be or not to be";
```

С-строки

Для задания строки можно использовать указатель на символьный тип.

```
1 char *m3;
```

В большинстве случаев для ее использования потребуется выделить и по окончанию работы освободить память.

```
1 #include <malloc.h>
2 ...
3 m3=(char*) malloc(10);
4 ...
5 free(m3);
```

В этом случае объявление массива переменной m3 может быть присвоен адрес массива:

```
1 m3 = m2;
2 *m3          эквивалентно m3[0]='T'
3 *(m3+1)      эквивалентно m3[1]='o'
```

Неправильно полагать, что в m3 теперь содержится копия m2. На самом деле этот указатель указывает не на копию, а на ту же самую строку. При любом изменении содержимого m3 изменяется m2. Однако, если именно это и требуется, то все в порядке. Здесь m2 является константой-указателем. Нельзя изменить m2, так как это означало бы изменение положения (адреса) массива в памяти, в отличие от m3.
Для указателя можно использовать операцию увеличения (перемещения на следующий символ):
m3++;

массив символьных строк

Иногда в программах возникает необходимость описание массива символьных строк. В этом случае можно использовать индекс строки для доступа к нескольким разным строкам.

```
1 char *poet[4] = {"Погиб поэт!", "- невольник↩
    чести -",
2 "Пал," , "оклеветанный молвой..."};
```

В этом случае poet является массивом, состоящим из четырех указателей на символьные строки. Каждая строка символов представляет собой символьный массив, поэтому имеется четыре указателя на массивы. Указатель poet[0] ссылается на первую строку:

```
1 *poet[0] эквивалентно 'П',
2 *poet[1] эквивалентно '-'.
```

С-строки. Ввод с консоли

```
1 char word[21];
2
3 printf("Введите слово: ");
4 scanf("%20s", word);
```

Три ключевые особенности ввода:

1. "**%20s**"у **scanf** означает ввод строки (до первого пробела или переноса), с ограничением максимальной длины в 19 символов (байт). Хотя размер введённой строки может быть и меньше
2. Символ окончания строки проставляется функцией по умолчанию (если длина группы символов более 20 штук, то в **phrase** попадёт только 19 из них и двадцатым будет поставлен символ окончания строки)
3. Поскольку переменная-массива **phrase** фактически хранит адрес первого элемента, знак **&** в **scanf** указывать не нужно

С-строки. Ввод с консоли

Безопасный ввод строки

`fgets(str, max_chars, stream_pointer)`

1. **str** - массив типа **char**, куда будут сохранена вводимая строка
2. **max_chars** - целое число, обозначающее **максимальное число символов**, которые будут сохранены в **str**
3. **stream_pointer** - указатель на структуру, ассоциированную с вводом текста. Для терминала это всегда переменная **stdin**, которая определена в **<stdio.h>**

```
1 const size_t SZ = 140;
2 ...
3 char phrase[SZ];
4
5 printf("Введите фразу: ");
6 fgets(phrase, 140, stdin);
```

Пример

Пример Посчитать количество введенных символов во введенной строке.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 int main() {
5   char s[80], sym;
6   int count, i;
7   system("chcp 1251");
8   system("cls");
9 // system("chcp 1251>null");
10   printf("Введите строку : ");
11   fgets(s,80,stdin);
12   printf("Введите символ : ");
13   sym = getchar();
14  for (i = 0; s[i] != '\0'; i++)
15   {
```

Пример

```
1   count = 0;
2   for (i = 0; s[i] != '\0'; i++)
3   {
4     if (s[i] == sym)
5       count++;
6   }
7   printf("В строке\n");
8   puts(s);        // Вывод строки
9   printf("символ ");
10  putchar(sym); // Вывод символа
11  printf(" встречается %d раз", count);
12  return 0;
13 }
```

функция fgets часто используется для ввода любых данных с клавиатуры в виде строки с целью дальнейшего преобразования функцией sscanf к нужному формату:

```
1 #include <string.h>
2 #include <stdio.h>
3 #include <conio.h>
4 int main()
5 { char s[50]; int x, err,i;
6 do
7  { printf("\n Введите целое число ");
8    fgets(s,50,stdin);
9    err=sscanf(s, "%d",&x);
10    if (err!=1) printf("\n Ошибка ввода");
11  } while (err!=1);
12   printf("\n Введено целое число %d \n", x);
13 sscanf("hello 1 2 3 4 5", "%s%d", s, &i);
14 printf("%s \n %d", s, i);
15 return 0;
16 }
```

можно использовать функции sprintf и sscanf для преобразования строк.

```
1 char str[50];
2 int i=15;
3 int j;
4 sprintf(str, "%d", i); // Записать в str ↩
    строковое представление i
5 sscanf(str, "%d", &j); // Записать в j число↩
    , содержащееся в строке str
6 sprintf(str, "i=%d and j=%d", i, j);
7 // содержимое str: "i=15 and j=15"
```

Хотя sprintf и sscanf довольно удобны, у них есть несколько недостатков. Во-первых, они не всегда быстро работают, во-вторых не типобезопасны. Например, если в строке формата вы укажите, что передаете два целых, а вместо этого передадите два double, ошибка обнаружится только при выполнении программы и найти ее причину будет не так-то просто.

Работа с текстом в C с помощью стандартной библиотеки

Работа с текстом. Стандартная библиотека

Библиотека языка C для работы со строками:

```
1 #include <string.h>
```

Получение длины строки **в байтах**

`size_t strlen(const char* str)`

```
1  char text[] = "A string";
2  // На экране покажется число 8
3  printf("Длина строки: %lu", strlen(text));
4
5  char text2[] = "Строка на русском";
6  /*
7     А здесь — зависит от ОС и её локальных
8     настроек. В русскоязычных Windows 7, 8,
9     10 — скорее всего покажет длину равную 17.
10    В современных ОС, основанных на Linux,
11    наиболее вероятное значение — 32
12 */
13 printf("Длина строки: %lu", strlen(text2));
```

Посимвольное сравнение строк

```
int strcmp(const char *first,
           const char *second)
```

Функция возращает:

1. значение $<$ **0** - первый несовпадающий символ в строке **first** меньше, чем в строке **second**;
2. значение $==$ **0** - все символы в обоих строках совпадают;
3. значение $>$ **0** - первый несовпадающий символ в строке **first** больше, чем в строке **second**;

Сравнение идёт с начала строк и до тех пор, пока в одной из них не встретится символ окончания строки. Фактически сравниваются целочисленные коды каждого символа (байта). stricmp сравнивает строки без учета регистра. Однако, все это относится только к латинице.

Сравнение строк

```
1  char str1[] = "Единица",
2       str2[] = "единица",
3       str3[] = "Единица";
4
5  printf("Сравнение str1 и str2: ");
6  printf("%d", strcmp(str1, str2));
7  printf("%d", stricmp(str1, str2));
8  printf("%d", stricmp("ONE", "one"));
9  printf("Сравнение str1 и str3: ");
10 printf("%d", strcmp(str1, str3));
```

Сравнение строк

```
1 char key_color[] = "оранжевый", answer[50];
2
3 do {
4   printf("\nУгадайте цвет: ");
5   scanf("%49s", answer);
6 } while ( strcmp(key_color, answer) != 0 );
7
8 puts("Правильный ответ!");
```

Возможный вывод программы:

```
Угадайте цвет: красный
Угадайте цвет: жёлтый
Угадайте цвет: оранжевый
Правильный ответ!
```

Посимвольное сравнение начальных фрагментов строк

```
int strncmp(const char *first,
            const char *second,
            size_t count)
```

Сравнение идёт с начала и до тех пор, пока в одной из строк не встретится символ окончания строки, **либо** не закончится сравнение **count** символов из каждой.

Посимвольное сравнение начальных фрагментов строк

```
1  char str1[] = "12 причин для роста",
2       str2[] = "12 отговорок от учёбы",
3
4  if ( strncmp(str1, str2, 2) == 0 ) {
5    puts("Обе строки начинаются с числа 12.");
6  }
7
8  // Как переносимым образом сравнивать
9  // фрагменты с текстом, отличным от ASCII
10 if ( strncmp(str1, str2, strlen("12 причин")↩
     ) != 0 ) {
11   puts ("Но не с 12 причин.")
12 }
```

НЕ ПРАВИЛЬНОЕ Копирование строки

```
1 char str1[10], str2[10];
2 str1="Hello";
3 str2=str1;
```

Одна и та же ошибка в обоих операторах =.
Имя массива нельзя использовать в левой части оператора присваивания.

Копирование строки ПРАВИЛЬНО

`char* strcpy(char *dest, const char *source)`

- Функция копирует строку **source** в строку **dest**
- Функция возращает указатель на строку **dest**. Как правило, возращаемое значение игнорируется
- Копируются **все** символы из строки, включая и завершающий *символ окончания строки*
- Следить за размером строк следует самостоятельно!

```
1 char source[] = "Hello!";
2 char dest[70];
3 strcpy(dest, source);
4
5 printf("Строка из dest: %s\n", dest);
6 printf("А её длина: %lu", strlen(dest));
7
8 strcpy(dest, "Hello");
```

Копирование фрагмента строки

```
char* strncpy(char        *dest,
              const char *source,
              size_t     count)
```

- **count** - целое безнаковое число, количество символов которое копируется в **dest**
- Если длина копируемой строки меньше, чем указано в аргументе **count**, то оставшиеся символы (байты) заполняются нулями
- Если длина копируемой строки больше, чем указано в аргументе **count**, то в строку **dest** не добавляется символ окончания '\0'

Копирование фрагмента строки

```
1 char source[] = "Просто длинная строка";
2 const size_t LEN = strlen("Просто длин");
3
4 char dest[LEN];
5 strncpy(dest, source, LEN);
6
7 printf("Фрагмент строки: %s", dest);
```

Добавление одной строки в другую (склеивание строк)

```
char* strcat(char *dest, const char *source)
```

- Функция добавляет строку **source** в конец строки **dest**
- Функция возращает указатель на строку **dest**. Как правило, возращаемое значение игнорируется
- Завершающий *символ окончания строки* переносится в конец объединённой строки
- Следить за размером строк - самостоятельно

Добавление одной строки в другую

```
1 char message[140];
2
3 strcat(message, "Эти строки ");
4 strcat(message, "были объеденены ");
5 strcat(message, "с помощью четырёх ");
6 strcat(message, "вызовов функции strcat.");
7
8 puts(message);
```

Добавление начального фрагмента одной строки в другую

```
char* strncat(char        *dest,
              const char *source,
              size_t      count)
```

- Функция добавляет **count** символов из строки **source** в конец строки **dest**
- Функция возращает указатель на строку **dest**. Как правило, возращаемое значение игнорируется
- Завершающий *символ окончания строки* переносится в конец объединённой строки
- Если длина строки **source** меньше, чем аргумент **count**, то добавляются только символы до '\0'

Добавление начального фрагмента одной строки в другую

```
1  char part1[] = "Здесь должно быть ",
2       part2[] = "сообщение не доставлено";
3
4  char str[50];
5
6  strcat(str, part1);
7  strncat(str, part2, strlen("сообщение"));
8
9
10 printf("Итоговая строка: \"%s\"", str);
```

Поиск первого вхождения символа в строке

```
char* strchr(const char *str, int character);
```

- Функция ищет символ **character** в строке **str**
- Функция возращает указатель на **первый** найденный символ
- Если совпадения не найдено, возращается **NULL**
- Несмотря на то, что второй аргумент передаётся как значение типа **int**, фактически внутри функции оно приводится к типу **char**
- Из предыдущего пункта следует *правило хорошего тона*: **strchr** должна использоваться **только** для символов из кодировки ASCII

Поиск первого вхождения символа в строке

```
1  char text[] = "Скажи-ка, дядя, ведь не даром\n"
2                "Москва, спалённая пожаром,\n"
3                "Французу отдана?\n"
4                "Ведь были ж схватки боевые,\n"
5                "Да, говорят, ещё какие!\n"
6                "Недаром помнит вся Россия\n"
7                "Про день Бородина!\n";
8  printf("Ищем все запятые в тексте\n%s\n", text);
9  puts("-----------------------------------");
10
11 char *p_space = strchr(text, ',');
12 while ( p_space != NULL) {
13   int place = p_space - text + 1;
14   printf("',' найдена на %d месте\n", place);
15   // Поиск продолжается с первого символа,
16   // идущего после ','
17   p_space = strchr(p_space + 1, ',');
18 }
```

Поиск подстроки (фрагмента)

```
char* strstr(const char *where,
             const char *what);
```

- **where** - строка, внутри которой ищется фрагмент **what**
- Если совпадение найдено, то есть фрагмент **what** присутствует в **where**, возращается указатель на его первый символ. Указатель содержит адрес элемента из **исходной строки**
- Если совпадения не найдено (фрагмент **what** не присутствует в строке **where**), возращается **NULL**

Поиск подстроки: замена слова "Что"на некоторое количество символов решётки (#)

```
1  char text[] = "Что делать? Что знать? Что ↩
       получится?";
2  const size_t SZ = strlen("Что");
3  char sharps[SZ + 1];
4  for (size_t i = 0; i < SZ; ++i) {
5    sharps[i] = '#';
6  }
7  sharps[SZ] = '\0';
8
9  printf("Исходный текст: \n%s\n\n", text);
10 char *p_str = strstr(text, "Что");
11 while (p_str != NULL) {
12   strncpy(p_str, sharps, SZ);
13   p_str = strstr(p_str + SZ + 1, "Что");
14 }
15 printf("Итоговый текст: \n%s\n\n", text);
```